Lab 15:

Introduction to Transfer Learning

In our previous lab, we built a Convolutional Neural Network (CNN) from scratch. While effective for simpler tasks, training deep networks from scratch requires massive datasets and significant computational power.

Transfer Learning allows us to take a model that has already been trained on a massive dataset (like ImageNet, which contains millions of images across 1000 categories) and repurpose it for our own specific task (like detecting malaria in cell images).

Key Concepts:

1. **Pre-trained Model (The Base):** We will import **VGG16**, a famous and powerful deep CNN architecture. We will load the weights it learned from the ImageNet dataset.
2. **Importing the Model:** PyTorch provides built-in modules (`torchvision.models`) to download these architectures.
3. **Freezing Layers:** We "freeze" the base model (`requires_grad = False`) to keep its weights intact while we train only our new custom head.
4. **Replacing the Head:** Pre-trained models are typically designed to output 1000 classes. We will replace the final classification layers (the "head") with our own layers designed for binary classification.
5. **Fine-Tuning:** Once our custom head is trained, we can optionally "unfreeze" the top few layers of the base model and train the whole system with a very low learning rate.

Task 1: Import Necessary Libraries

Import PyTorch, Torchvision, and other necessary libraries for data loading and visualization.

In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import os


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cpu
```

In [ ]:

```
import os
import urllib.request
import zipfile
import random
import shutil


dataset_url = "https://data.lhncbc.nlm.nih.gov/public/Malaria/cell_images.zip"
zip_path = "cell_images.zip"
original_dataset_dir = "cell_images"
base_dir = "./Malaria_Dataset"


if not os.path.exists(zip_path) and not os.path.exists(base_dir):
    print("Downloading Malaria dataset. Please wait...")
    urllib.request.urlretrieve(dataset_url, zip_path)
    print("Download complete!")


if not os.path.exists(original_dataset_dir) and not os.path.exists(base_dir):
    print("Extracting files...")
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(".")
    print("Extraction complete!")


if not os.path.exists(base_dir):
    print("Creating train/val split structure...")
    classes = ['Parasitized', 'Uninfected']

    # Create the folder structure
    for split in ['train', 'val']:
        for cls in classes:
            os.makedirs(os.path.join(base_dir, split, cls), exist_ok=True)


    split_ratio = 0.8
    for cls in classes:
        src_dir = os.path.join(original_dataset_dir, cls)
        filenames = os.listdir(src_dir)


        filenames = [f for f in filenames if f.endswith('.png')]


        random.shuffle(filenames)


        split_idx = int(len(filenames) * split_ratio)
        train_files = filenames[:split_idx]
        val_files = filenames[split_idx:]


        for f in train_files:
            shutil.copy(os.path.join(src_dir, f), os.path.join(base_dir, 'train'


        for f in val_files:
            shutil.copy(os.path.join(src_dir, f), os.path.join(base_dir, 'val',
```

```
    print("Dataset successfully organized into ./Malaria_Dataset/train and ./Mal
else:
    print("Dataset already downloaded and organized!")
```

PART 1

In [ ]:
```
print("Loading VGG16...")
vgg16_full = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
vgg16_full = vgg16_full.to(device)
vgg16_full.eval()
```

In [ ]:
```
labels_url = "https://raw.githubusercontent.com/anishathalye/imagenet-simple-lab
imagenet_labels = json.loads(requests.get(labels_url).text)
```

In [ ]:
```
import io
import requests # Ensure requests is imported

# Download a sample image of a dog from a reliable source
img_url = "https://raw.githubusercontent.com/pytorch/vision/main/gallery/assets/

response = requests.get(img_url)

# Check if the request was successful
if response.status_code != 200:
    raise Exception(f"Failed to download image from {img_url}. Status code: {res


content_type = response.headers.get('Content-Type', '')
if not content_type.startswith('image'):
    raise Exception(f"URL content is not an image. Content-Type: {content_type}"

img = Image.open(io.BytesIO(response.content))

plt.imshow(img)
plt.axis('off')
plt.title("Input Image")
plt.show()
```

PART 2

Task 2: Data Augmentation and Loading

Initialize transformations (rescaling to 0-1 via `ToTensor()`, resizing, and adding random augmentations). Use `datasets.ImageFolder` and `DataLoader` to load the Train and Test sets.

In [ ]:
```
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

In [ ]:
```
train_transforms = transforms.Compose([
    transforms.Resize((224,224)),
```

```
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ToTensor()
])
```

In [ ]:
```
test_transforms = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor()
])
```

In [ ]:
```
train_dataset = datasets.ImageFolder(
    root='dataset/train',
    transform=train_transforms
)
```

In [ ]:
```
test_dataset = datasets.ImageFolder(
    root='dataset/test',
    transform=test_transforms
)
```

In [ ]:
```
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)
```

In [ ]:
```
test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False
)
```

In [ ]:
```
print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))
```

Task 3: Load Base Model and Freeze Layers

Load the pre-trained VGG16 model. Iterate through its parameters and turn off gradients so the weights don't update during the initial training phase.

In [ ]:
```
import torchvision.models as models
# Load pre-trained VGG16
vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)

for param in vgg16.parameters():
    param.requires_grad = False
```

Task 4: Add Custom Head (Classifier)

Replace `vgg16.classifier` with a new `nn.Sequential` block containing a Linear layer (e.g., 256 units), a ReLU activation, and a final Linear layer for our binary output.

In [ ]:
```
import torch.nn as nn
num_features = vgg16.classifier[0].in_features
```

```
vgg16.classifier = nn.Sequential(
    nn.Linear(25088, 256),
    nn.ReLU(),
    nn.Linear(256, 2)
)

vgg16 = vgg16.to(device)

print(vgg16)
```

Task 5: Define Loss Function and Optimizer

Define the criterion (Loss Function). Set up the optimizer (e.g., Adam) to *only* update the parameters of the new classifier we just added.

In [ ]:
```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(vgg16.classifier.parameters(), lr=0.001)

print(optimizer)
```

Task 6: Train the Model

Create the PyTorch training loop. Iterate over the epochs, perform forward passes, calculate the loss, backpropagate, and update the optimizer steps. Keep track of training and validation accuracy.

In [ ]:
```
def train():
    vgg16.train()

    num_epochs = 5
    running_loss = 0

    for epoch in range(num_epochs):

        correct = 0
        total = 0

        print(f"\n--- Epoch {epoch + 1}/{num_epochs} ---")

        for batch_No, (data, target) in enumerate(train_loader):

            output = vgg16(data)

            loss = criterion(output, target)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

            _, predicted = torch.max(output, 1)
```

```
            total += target.size(0)
            correct += (predicted == target).sum().item()

        train_accuracy = correct / total

        print(f"Epoch {epoch + 1} completed! Loss : {running_loss / len(train_lo
        print(f"Training Accuracy : {train_accuracy}")
```

Task 7: Plot History

Visualize the Training vs. Validation Accuracy and Loss using Matplotlib to check for overfitting or underfitting.

In [ ]:

Task 8: Fine Tuning (Optional)

Now that our custom head is trained, let's unfreeze the last convolutional block of VGG16. We will compile a new optimizer with a much **lower** learning rate so we don't drastically alter the pre-trained weights, and retrain the model for a few more epochs.

In [ ]: